

GLOBELENS AI

Comprehensive Architecture Specification & Engineering Blueprint

VERSION 2.0 • TECHNICAL DOCUMENTATION ARTIFACT

1. Executive Product Vision

GlobeLens AI is an advanced intelligence platform engineered to revolutionize the modern news consumption paradigm. In an era dominated by hyper-fragmented reporting, systemic misinformation, and editorial bias, the platform serves as an automated central node that processes, cross-references, cleans, and contextualizes global news feeds in real-time.

1.1 Core Problems Addressed

- **Redundancy Fatigue:** Modern readers waste hours scanning multiple digital outlets, repeatedly consuming identical background contexts just to uncover single unique details or peripheral developments.
- **Trust Uncertainty:** Audiences lack immediate metrics to evaluate how many independent, high-credibility media entities corroborate or conflict with a given breaking journalistic claim.
- **Context Collapse:** News updates routinely appear in isolation. Crucial historical paths, geographic ties, and evolution trajectories are stripped away, causing a fractured understanding of shifting scenarios.

1.2 High-Level Architectural Solutions

The platform orchestrates multi-source web scrapers, vector embeddings, cluster algorithms, and large language models (LLMs) to automatically collect articles and group them into cohesive, geo-tagged **Events**. This technical foundation enables a suite of intelligent user capabilities:

Key Functional Offerings:

1. **AI Synthesis:** Collates data across international layers (e.g., BBC, CNN, Al Jazeera) and synthesizes them into an objective summary with line-by-line traceability.
2. **Reliability Metrics & Bias Detection:** Dynamically calculates factual consistency and highlights the socio-political lean of sources using real-time algorithmic classification.
3. **Dual-View Interface:** Seamlessly bridges a classic linear editorial news feed with a immersive, geolocation-anchored map view.
4. **Deep-Dive Timelines:** Automatically links chronological threads tracking international stories back to their day-of-origin context.

2. Actor Roles & Authorization Matrix

To ensure strict operational boundaries and system integrity, GlobeLens AI enforces four distinct user profiles. Access rights are modeled via unified permission states mapping directly across controllers:

FEATURE CAPABILITY	GUEST	AUTH. USER	JOURNALIST	ADMIN
Read home feed & timelines	✓	✓	✓	✓
Interactive map & Geolocation feeds	✓	✓	✓	✓
View reliability scores & media bias lean	✓	✓	✓	✓
Write community comments / tracking	—	✓	✓	✓
Submit external URLs for AI Fact-Checking	—	✓	✓	✓
Subscribe to custom topic/country alerts	—	✓	✓	✓
Submit, edit, and manage custom articles	—	—	✓	✓
Access System Dashboard & Analytics	—	—	—	✓
Moderate comments, block users, mask items	—	—	—	✓

3. Data Pipeline & System Flow

The backend architecture relies on an asynchronous data ingestion and transformation pipeline to process stories from ingestion to public visibility:

PIPELINE STEP	INVOLVED COMPONENTS	DETAILED ARCHITECTURAL OPERATION
1. Ingestion	ScraperService, ArticleRepository	Scrapes RSS feeds/media APIs, extracts raw metadata, normalizes schema, and persists state with status SCRAPED .
2. Vectorization	EmbeddingService, Redis Cache	Extracts text blocks, calls embedding models to produce vector spaces, stores vectors in PostgreSQL/pgvector, flags as EMBEDDED .
3. Clustering	ClusteringService, EventService	Computes similarity matrices across unassigned items. Links articles together into new or existing Events based on Euclidean/Cosine distance thresholds. Status changes to CLUSTERED .
4. LLM Synthesis	LLMService, EventRepository	Feeds clustered facts into Claude/LLM models to extract geographic coordinates, isolate primary focus topics, and build the authoritative consolidated summary. Status finalized to PROCESSED .

4. Domain Data Model Blueprint

The relational and vector database structure maps entities tightly to provide rich analytics while decoupling compute-heavy tasks:

- **User:** Manages authentication identities and preference arrays (`preferredTopics`, `preferredCountries`) to handle notifications. Supports administrative security flags like `isBlocked`.
- **Source:** Models media publishers. Tracks systemic criteria such as calculated `credibilityScore` and AI-generated `biasLean` alignments (`Left to Right` enums).
- **Article:** The foundational unit of content. Contains URLs, timestamps, publication fields, and visibility control flags (`isHidden`) for moderation filters.
- **Event:** High-level aggregations containing clusters of Articles. Stores central AI-generated summaries, importance weights, coordinates (latitude, longitude), and promotional administrative markers.
- **Embedding:** Highly decoupled storage referencing `articleId` alongside raw vector arrays to handle lightning-fast similarity scoring without impacting core domain entities.
- **FactCheckRequest:** Captures user-submitted external verification attempts, storing status results (`True`, `False`, `Uncertain`) alongside descriptive natural-language explanations.

5. Loose Coupling Design & Interface Separation

To prevent structural rigidity and enable the system to satisfy the Dependency Inversion Principle, concrete application services do not link directly to repositories or infrastructure clients. Instead, they operate over strict abstraction boundaries:

Decoupling Engineering Benefits:

By ensuring controllers route requests through interfaces (e.g., `IUserService`, `IArticleService`) which interact with generic interface definitions (e.g., `UserRepository`), the core business engine remains completely agnostic of the underlying database technology. Swapping PostgreSQL for MongoDB or shifting from Elasticsearch to an alternate provider demands zero alterations inside the core domain logic layer. Furthermore, it allows clean testing execution via mock injections.

6. Production-Ready PlantUML Architecture Schema

Below is the exhaustive, revised, loose-coupled class diagram specification for GlobeLens AI, accommodating all system requirements including Journalist management capabilities, Audio Listening endpoints, Bias Detection attributes, and explicit interface decouplings:

```
@startuml
skinparam packageStyle rectangle
skinparam linetype ortho

package "Enums" {
    enum ProcessingStatus {
        SCRAPED
        EMBEDDED
    }
}
```

```

        CLUSTERED
        PROCESSED
    }
    enum UserRole {
        GUEST
        AUTH_USER
        JOURNALIST
        ADMIN
    }
    enum FactCheckResult {
        TRUE
        FALSE
        UNCERTAIN
    }
    enum BiasLean {
        LEFT
        CENTER_LEFT
        CENTER
        CENTER_RIGHT
        RIGHT
    }
}

package "Controllers" {
    class SearchController {
        +GET /search?q=
        +GET /search/suggestions?q=
    }
    class CommentController {
        +POST /events/{id}/comments
        +GET /events/{id}/comments
        +PUT /comments/{id}
        +DELETE /comments/{id}
    }
    class EventController {
        +GET /events
        +GET /events/{id}
        +GET /events/trending
        +GET /events/latest
        +GET /events/country/{country}
        +GET /events/topic/{topic}
        +GET /events/map
        +POST /events/{id}/follow
    }
    class AdminController {
        +GET /admin/dashboard
        +GET /admin/users
        +PUT /admin/users/{id}/role
        +PUT /admin/users/{id}/block
        +DELETE /admin/users/{id}
        +DELETE /admin/comments/{id}
        +POST /admin/events/promote
        +PUT /admin/articles/{id}/hide
        +POST /admin/sources
        +GET /admin/stats
    }
    class FactCheckController {
        +POST /fact-check
        +GET /fact-check/{id}
        +GET /users/{id}/fact-checks
    }
    class UserController {
        +GET /users/{id}
        +PUT /users/{id}
        +DELETE /users/{id}
    }
}

```

```

    }
    class AuthController {
        +POST /auth/register
        +POST /auth/login
        +GET /auth/me
        +POST /auth/logout
        +POST /auth/refresh
    }
    class ArticleController {
        +GET /articles/{id}
        +GET /events/{id}/articles
        +POST /articles
        +PUT /articles/{id}
        +DELETE /articles/{id}
        +GET /articles/{id}/audio
    }
}

package "Application Services" {
    class ScraperService {
        +scrapeSource()
        +normalizeArticle()
        +saveArticle()
    }
    class UserService {
        +registerUser()
        +getUserById()
        +updateUser()
        +deleteUser()
        +blockUser()
        +updatePreferences()
    }
    class AuthService {
        +register()
        +login()
        +hashPassword()
        +verifyPassword()
        +generateAccessToken()
        +verifyToken()
    }
    class EmbeddingService {
        +generateEmbedding(article)
        +storeEmbedding()
    }
    class CacheService {
        +get()
        +set()
        +invalidate()
    }
    class ClusteringService {
        +computeSimilarity()
        +clusterArticles()
        +assignEvent()
    }
    class EventService {
        +createEvent()
        +updateEvent()
        +calculateImportance()
        +promoteEvent()
    }
    class FactCheckService {
        +analyzeClaim()
        +generateExplanation()
    }
    class SearchService {

```

```

        +indexEvent()
        +searchEvents()
        +getSuggestions()
    }
    class LLMService {
        +generateSummary()
        +generateTitle()
        +detectCountry()
        +extractTopic()
        +detectBias()
    }
    class AudioService {
        +textToSpeech(articleId)
    }
    class NotificationService {
        +sendNotification(userId, message)
        +subscribeToEvent(userId, eventId)
        +subscribeToTopic(userId, topic)
    }
}

package "Domain Entities" {
    class Source {
        +id: UUID
        +name: String
        +credibilityScore: Float
        +country: String
        +biasLean: BiasLean
    }
    class Article {
        +id: UUID
        +title: String
        +content: Text
        +url: String
        +publishedAt: DateTime
        +createdAt: DateTime
        +processingStatus: ProcessingStatus
        +sourceId: UUID
        +eventId: UUID
        +isHidden: Boolean
    }
    class Event {
        +id: UUID
        +title: String
        +summary: Text
        +topic: String
        +country: String
        +latitude: Float
        +longitude: Float
        +importanceScore: Float
        +isPromoted: Boolean
        +createdAt: DateTime
        +updatedAt: DateTime
    }
    class User {
        +id: UUID
        +name: String
        +email: String
        +password: String
        +role: UserRole
        +isBlocked: Boolean
        +preferredTopics: List
        +preferredCountries: List
    }
    class Comment {

```

```

        +id: UUID
        +content: Text
        +createdAt: DateTime
        +userId: UUID
        +eventId: UUID
    }
    class FactCheckRequest {
        +id: UUID
        +inputTextUrl: String
        +result: FactCheckResult
        +explanation: Text
        +createdAt: DateTime
        +userId: UUID
    }
    class Embedding {
        +id: UUID
        +vector: Vector
        +model: String
        +createdAt: DateTime
        +articleId: UUID
    }
}

package "Repositories" {
    class SearchRepository {
        +index()
        +search()
    }
    class FactCheckRepository {
        +save()
        +findById()
        +findByUser()
    }
    class EventRepository {
        +save()
        +findById()
        +findTrending()
    }
    class UserRepository {
        +save()
        +findById()
        +findByEmail()
        +update()
    }
    class ArticleRepository {
        +save()
        +findById()
        +findUnprocessed()
        +findByEvent()
    }
    class CacheRepository {
        +get()
        +set()
    }
    class SourceRepository {
        +save()
        +findById()
        +findAll()
    }
}

package "Infrastructure" {
    class LLMPProvider
    class Elasticsearch
    class PostgreSQL

```

```
class Redis
}

Source "1" -- "0..*" Article : publishes
Event "1" -- "0..*" Article : contains
Event "1" -- "0..*" Comment : groups
User "1" -- "0..*" Comment : writes
User "1" -- "0..*" FactCheckRequest : creates
Article "1" -- "1" Embedding : has

Controllers --> "Application Services"
"Application Services" --> Repositories
Repositories --> Infrastructure
@endum
```